

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1441
Design

COURSE NAME: Compiler

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 20%

QUESTIONS: 3
Mode: Take-Home

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

*I __K. Madhulika (192524190)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **Madhulika**

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	

	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100
--	------	------	------	------	------	-------

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Q1. Hospital Patient Billing System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

A hospital billing application calculates consultation fees, medicine charges, and total bill amount.

Task

1. Write declarations for all variables.
2. Convert the following high-level code into three-address intermediate code:
 $\text{bill} = \text{consultation} + \text{medicine}$
 $\text{tax} = \text{bill} * \text{gst}$
 $\text{final_bill} = \text{bill} + \text{tax}$
3. Optimize the intermediate code (if possible).

Q2. Library Fine Calculation System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A library system checks whether a student should pay a fine based on:

- $\text{Days_Late} > 10$
- $\text{Book_Damaged} == \text{true}$

Task

1. Convert the condition into Boolean expression intermediate code.
2. Generate flow of control using backpatching technique.
3. Represent using labelled instructions.

Q3. Railway Reservation Status System

Concepts Covered: Case Statements

Problem Statement

A railway reservation system displays ticket status based on input:

```
switch(status)
case 1: CONFIRMED
case 2: RAC
case 3: WAITING
```

Task

1. Convert the case statement into intermediate code.
2. Use jump tables or conditional jumps.
3. Explain efficiency of your approach.

Q4. University Result Processing System

Concepts Covered: Procedure Calls

Problem Statement

A university system processes results using functions:

calculateMarks()

calculateGrade()

publishResult()

Task

1. Generate intermediate code for procedure calls.
2. Show:
 - o Parameter passing
 - o Call and return sequence
3. Use stack-based representation.

Q5. Weather Monitoring System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A weather monitoring system activates alert if:

- Temperature > 40 AND Humidity > 80

Task

1. Write Boolean expressions.
2. Convert into intermediate code with short-circuit evaluation.
3. Apply backpatching to handle jumps.
4. Assign final output to variable alert_status.

Q6. Airline Ticket Fare System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

An airline system calculates total ticket fare including luggage charges and service tax.

Task

1. Write declarations for all variables.
2. Convert the following code into three-address code:
fare = basefare + luggage
tax = fare * servicetax
totalfare = fare + tax

3. Optimize the generated code.

Q7. Employee Attendance System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

An employee is marked eligible for bonus if:

- Attendance > 90
- Performance > 8

Task

1. Convert the condition into Boolean expression intermediate code.
2. Generate labelled control flow instructions.
3. Apply backpatching technique.

Q8. Smart Parking Management System

Concepts Covered: Case Statements

Problem Statement

A parking system allocates slots based on vehicle type:

```
switch(vehicle)
case 1: BIKE
case 2: CAR
case 3: TRUCK
```

Task

1. Convert the switch statement into intermediate code.
2. Use conditional jumps.
3. Explain time efficiency of the generated code.

Q9. ATM Transaction Processing System

Concepts Covered: Procedure Calls

Problem Statement

An ATM system performs operations using functions:

```
checkBalance()
withdrawCash()
printReceipt()
```

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing mechanism.
3. Use stack-based representation.

Q10. Industrial Safety Monitoring System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A factory alarm turns ON if:

- `GasLevel > 70 AND FireDetected == true`

Task

1. Write Boolean expressions.
2. Generate intermediate code using short-circuit evaluation.
3. Apply backpatching technique.
4. Assign final result to `alarm_status`.

Q11. Mobile Recharge Billing System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

A mobile recharge system calculates recharge amount, tax, and final payable amount.

Task

1. Declare all variables.
2. Convert the following statements into TAC:
 `amount = recharge + addon`
 `gst = amount * tax`
 `payable = amount + gst`
3. Optimize the intermediate representation.

Q12. Online Examination Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A student can attend the exam if:

- `Attendance >= 75`
- `FeePaid == true`

Task

1. Generate Boolean expression TAC.
2. Show control flow using labels.
3. Apply backpatching.

Q13. Metro Train Route System

Concepts Covered: Case Statements

Problem Statement

A metro system selects route based on route number:

```
switch(route)
case 1: REDLINE
case 2: GREENLINE
case 3: BLUELINE
```

Task

1. Generate intermediate code for switch statement.
2. Use labelled instructions.
3. Analyze execution efficiency.

Q14. Food Delivery Application

Concepts Covered: Procedure Calls

Problem Statement

A food delivery app performs operations using:

verifyOrder()

calculateBill()

sendNotification()

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing and return mechanism.
3. Represent using stack model.

Q15. Smart Irrigation Control System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

An irrigation system turns ON water supply if:

- SoilMoisture < 30 AND Temperature > 35

Task

1. Write Boolean expression.
2. Convert into intermediate code.
3. Apply short-circuit evaluation and backpatching.
4. Store result in water_status.

Q16. Electricity Bill Generation System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

An electricity board calculates energy charge, surcharge, and total payable amount.

Task

1. Write declarations for all variables.
2. Convert the following code into TAC:
charge = units * rate
surcharge = charge * tax
bill = charge + surcharge
3. Optimize the generated intermediate code.

Q17. Scholarship Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A student is eligible for scholarship if:

- $CGPA > 8.5$
- $Income < 200000$

Task

1. Convert the condition into Boolean TAC.
2. Generate labelled flow of control.
3. Apply backpatching technique.

Q18. Smart Elevator Control System

Concepts Covered: Case Statements

Problem Statement

An elevator system selects floor operations based on input:

```
switch(floor)
case 1: OPEN_DOOR
case 2: CLOSE_DOOR
case 3: MOVE_UP
```

Task

1. Convert switch-case into intermediate code.
2. Use jump instructions.
3. Explain efficiency of the approach.

Q19. Online Banking Transaction System

Concepts Covered: Procedure Calls

Problem Statement

An online banking application performs functions:

```
authenticateUser()
transferMoney()
sendOTP()
```

Task

1. Generate intermediate code for procedure calls.
2. Show call-return sequence.
3. Use stack-based representation.

Q20. Smart Street Light Automation System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A smart street light turns ON if:

- `LightIntensity < 20 AND MotionDetected == true`

Task

1. Write Boolean expressions.
2. Generate intermediate code using short-circuit evaluation.
3. Apply backpatching for jumps.
4. Store final result in `light_status`.

Q21. Online Shopping Discount System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

An online shopping application calculates discount, tax, and final payable amount for customers.

Task

1. Declare all required variables.
2. Convert the following statements into TAC:
 `discount = price * rate`
 `netamount = price - discount`
 `finalamount = netamount + tax`
3. Optimize the intermediate code if possible.

Q22. Passport Verification System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A passport application is approved if:

- `Age >= 18`
- `Citizenship == true`

Task

1. Generate Boolean expression intermediate code.
2. Show flow of control using labels.
3. Apply backpatching technique.

Q23. Smart Classroom Device Control

Concepts Covered: Case Statements

Problem Statement

A smart classroom controls devices using:

```
switch(device)
case 1: PROJECTOR
```

case 2: FAN
case 3: LIGHT

Task

1. Convert the switch statement into intermediate code.
2. Use conditional jumps or jump tables.
3. Explain the efficiency of the generated code.

Q24. Hospital Appointment Management System
Concepts Covered: Procedure Calls

Problem Statement

A hospital system executes functions:
checkDoctorAvailability()
bookAppointment()
sendConfirmation()

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing and return sequence.
3. Use stack-based representation.

Q25. Forest Fire Detection System
Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A forest monitoring system activates emergency alert if:
● Temperature > 45 AND SmokeDetected == true

Task

1. Write Boolean expressions.
2. Convert into intermediate code with short-circuit evaluation.
3. Apply backpatching to control jumps.
4. Assign result to fire_alert.

Q26. Hotel Room Billing System
Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

A hotel management system calculates room charge, food bill, and total payable amount.

Task

1. Declare all variables.
2. Convert the following statements into TAC:
bill = roomcharge + foodcharge

```
gst = bill * tax
total = bill + gst
```

3. Optimize the intermediate code.

Q27. Vehicle Insurance Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A customer is eligible for insurance if:

- VehicleAge < 10
- LicenseValid == true

Task

1. Generate Boolean expression TAC.
2. Represent control flow using labels.
3. Apply backpatching technique.

Q28. Digital TV Channel Selection System

Concepts Covered: Case Statements

Problem Statement

A digital TV system selects channels using:

```
switch(channel)
case 1: SPORTS
case 2: NEWS
case 3: MOVIES
```

Task

1. Generate intermediate code for switch statement.
2. Use labelled jumps.
3. Explain execution efficiency.

Q29. Railway Ticket Booking System

Concepts Covered: Procedure Calls

Problem Statement

A railway reservation system performs functions:

```
checkSeatAvailability()
bookTicket()
sendTicket()
```

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing mechanism.
3. Use stack representation.

Q30. Smart Water Tank Monitoring System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A water pump turns ON if:

- $\text{WaterLevel} < 20$ AND $\text{MotorStatus} == \text{OFF}$

Task

1. Write Boolean expressions.
2. Generate intermediate code with short-circuit evaluation.
3. Apply backpatching.
4. Store final result in `pump_status`.

Q31. Supermarket Billing System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

A supermarket billing application calculates item cost, discount, and final bill amount.

Task

1. Declare all required variables.
2. Convert the following statements into TAC:
 $\text{cost} = \text{quantity} * \text{price}$
 $\text{discount} = \text{cost} * \text{rate}$
 $\text{finalbill} = \text{cost} - \text{discount}$
3. Optimize the intermediate code.

Q32. Online Voting Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A citizen is eligible to vote if:

- $\text{Age} \geq 18$
- $\text{IDVerified} == \text{true}$

Task

1. Generate Boolean expression intermediate code.
2. Show labelled control flow.
3. Apply backpatching technique.

Q33. Smart Traffic Lane System

Concepts Covered: Case Statements

Problem Statement

A traffic management system controls lanes using:

```
switch(lane)
case 1: LEFT
case 2: RIGHT
case 3: STRAIGHT
```

Task

1. Convert the switch statement into intermediate code.
2. Use jump instructions.
3. Explain efficiency of your method.

Q34. Online Library Management System

Concepts Covered: Procedure Calls

Problem Statement

A digital library system performs operations using:

searchBook()

issueBook()

updateRecord()

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing and return sequence.
3. Use stack-based representation.

Q35. Smart Energy Saving System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

An energy-saving system switches OFF appliances if:

- PowerUsage > 1000 AND RoomEmpty == true

Task

1. Write Boolean expressions.
2. Generate intermediate code using short-circuit evaluation.
3. Apply backpatching.
4. Assign result to energy_status.

Q36. Restaurant Billing System

Concepts Covered: Declarations, Assignment Statements, Intermediate Code

Problem Statement

A restaurant system calculates food cost, service charge, and total payable amount.

Task

1. Declare all variables.

2. Convert the following code into TAC:
 `foodbill = quantity * rate`
 `service = foodbill * tax`
 `total = foodbill + service`
3. Optimize the generated code.

Q37. College Admission Eligibility System

Concepts Covered: Boolean Expressions, Backpatching

Problem Statement

A student is eligible for admission if:

- `Marks >= 75`
- `EntranceQualified == true`

Task

1. Generate Boolean TAC.
2. Represent control flow using labels.
3. Apply backpatching technique.

Q38. Smart Home Lighting System

Concepts Covered: Case Statements

Problem Statement

A smart home controls lighting modes using:

```
switch(mode)
case 1: NIGHT
case 2: DAY
case 3: PARTY
```

Task

1. Convert switch-case into intermediate code.
2. Use conditional jumps.
3. Explain efficiency of generated code.

Q39. Online Medicine Delivery System

Concepts Covered: Procedure Calls

Problem Statement

An online medicine application performs functions:

```
verifyPrescription()
processOrder()
dispatchMedicine()
```

Task

1. Generate intermediate code for procedure calls.
2. Show parameter passing and return process.

3. Use stack-based representation.

Q40. Smart Security Door System

Concepts Covered: Boolean Expressions, Assignment, Backpatching

Problem Statement

A security door unlocks if:

- `FaceRecognized == true AND OTPVerified == true`

Task

1. Write Boolean expressions.
2. Generate intermediate code using short-circuit evaluation.
3. Apply backpatching to handle jumps.
4. Store final result in `door_status`.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretatio n of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
-------------------------------	-----	--	--	---	----------------------

CO4 AT2 - Industry Problem Solving

CSA 1419 - Compiler Design

Name: K. Madhulika

Reg no: 192524190

13. Metro Train Route System:

Problem:

A metro system selects a route based on the route number

switch (route)

case 1: RedLine

case 2: GreenLine

case 3: Blue Line

Intermediate code Generation:

1: if route == 1 goto 4
goto 8

2: if route == 2 goto 5
goto 8

3: if route == 3 goto 6
goto 8

4: print "REDLINE"
goto 8

5: print "GREENLINE"
goto 8

6: print "BLUELINE"
go to 8

7: print "Invalid Route"

8:

Use of Labelled instructions:

Labels identify the starting point of each route's instructions.

4: REDLINE

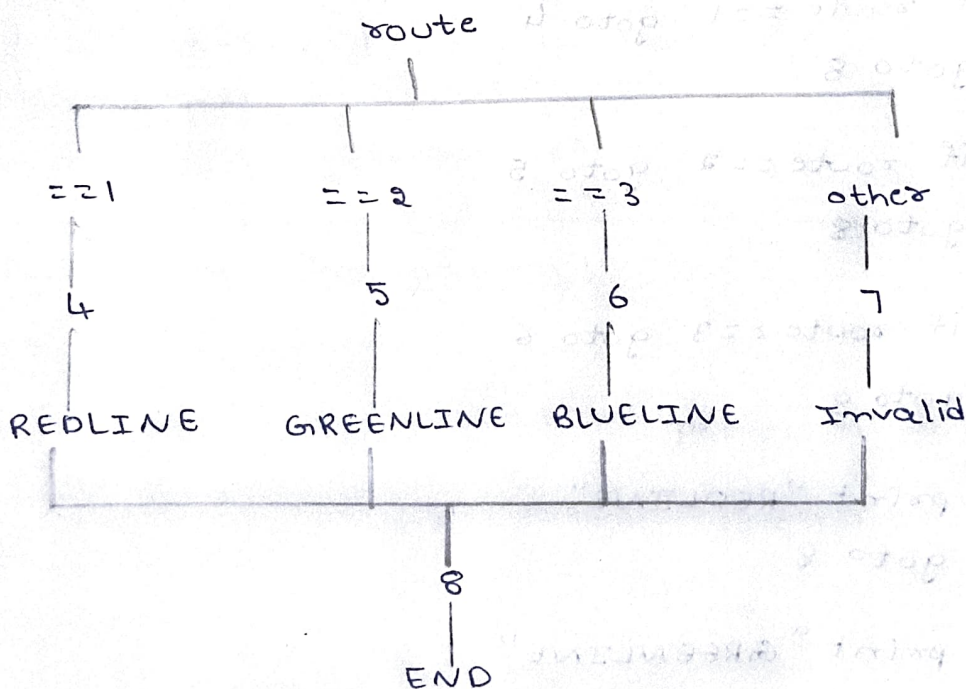
5: BLUELINE GREEN

6: BLUELINE

7: Invalid Route

8: End

The execution flow is:



Execution Efficiency Analysis:

The above implementation checks the cases sequentially.

route	comparisons required
1	1
2	2
3	3
invalid	3

Therefore, the best case is $O(1)$ and the worst case is $O(n)$, where n is the number of cases.

Result:

The switch statement is successfully translated into three-address code using labelled instructions. For a small number of cases, conditional jumps are efficient and easy to implement. For a large number of cases, a jump-table implementation provides better execution efficiency.

14. Food Delivery Application:

Problem Statement:

A food delivery application performs the following procedures:

verifyorder()

calculateBill()

sendNotification()

The compiler must generate intermediate code for these procedure calls and represent how parameters, return values and the stack are handled.

Intermediate code Generation:

Assume the application has:

VerifyOrder (order)

CalculateBill (order)

SendNotification (phone, message)

Three-Address code:

param order

call VerifyOrder, 1

param order

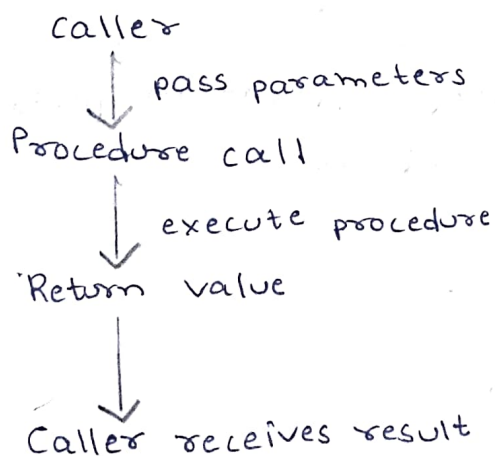
t1 = call CalculateBill, 1

param phone

param message

call SendNotification, 2

The execution flow is:



Stack Model Representation :

Return Address
Parameter: order
Local variables
Temporary values

Stack frame

Initially

Main Program

After calling `Verifyorder(order)`:

<code>verifyorder()</code> return address order
Main Program

Then `calculateBill(order)` is called:

<code>calculateBill()</code> return address order bill
Main Program

Result :

The procedure calls are successfully represented using Three-Address code. Parameters are

passed using param, procedures are invoked using call, and return values are stored in temporary variables. The stack model manages each procedure call through an activation record containing parameters, return address, local variables and temporary values.

15. Smart Irrigation Control System:

Problem:

The irrigation system turns ON the water supply when:

→ Soil moisture is less than 30, AND

→ Temperature is greater than 35.

Boolean Expression:

$(\text{soil-moisture} < 30) \text{ AND } (\text{temperature} > 35)$

In logical form:

$B = (\text{soil-moisture} < 30) \ \&\& \ (\text{temperature} > 35)$

AND operator means both conditions are true.

Intermediate code Generation:

1: if soil-moisture < 30 goto 2
goto 4

2: if temperature > 35 goto 3
goto 4

3: water-status = ON

goto 5

4: water-status = OFF

5:

Short circuit Evaluation:

For: A AND B

the second condition is evaluated only if the first condition is true.

Here: A = soil-moisture < 30

B = temperature > 35

So, if soil-moisture < 30

check temperature > 35

else

water-status = OFF

ex: soil-moisture = 25

soil-moisture = 25

temperature = 38

temperature = 30

water-status = ON

water-status = OFF

Back patching:

For the Boolean Expression:

$(\text{soil-moisture} < 30) \ \&\& \ (\text{temperature} > 35)$

We maintain:

True List $\rightarrow$ condition leading to 3

False list $\rightarrow$ Condition leading to 4.

Result :

The Boolean expression is successfully converted into intermediate code using short-circuit evaluation and Backpatching. The variable water-status is assigned ON only when soil moisture is below 30 and temperature is above 35, otherwise it is assigned OFF.